

LABORATORY PROJECT REPORT
SMART SURVEILLANCE SYSTEM
USING ESP32-CAM
EXPERIMENT 6

Section 1

Group 2

Semester 1 2025/2026

NO	NAME	MATRIC NO
1.	ADAM NURUDDIN BIN HELMI	2215783
2.	NUR FATIHAH HANNANI BINTI HASMAYADI	2227062
3.	NUR SYAHZMAN AZIQ BIN MOHD SHAHMADI	2310037

Date of submission: 19th November 2025

Abstract

This laboratory exercise explores the integration of Internet of Things (IoT) technology with mechatronic systems through the development of a Smart Surveillance System using the ESP32-CAM module. The system demonstrates wireless communication, actuator control, and real-time video processing by implementing live video streaming over a Wi-Fi network combined with servo motor-controlled horizontal camera panning. Students interface the ESP32-CAM with an SG90/MG90S servo motor to create a motorized camera base capable of continuous or manual motion control. The system architecture encompasses wireless data transmission using HTTP protocols, Pulse Width Modulation (PWM) for servo control, and optional sensor integration for motion-triggered surveillance. Through hands-on implementation, students gain practical experience in microcontroller programming, web server development, and the integration of multiple subsystems, including sensors, actuators, and communication modules. The lab progressively builds complexity from basic camera streaming to advanced features such as button-triggered motion control and optional face detection capabilities. This project provides foundational knowledge in IoT-based surveillance systems and demonstrates key mechatronic principles, including sensor-actuator coordination, embedded web services, and real-time system integration. The completed prototype serves as a functional demonstration of a cost-effective, remotely accessible surveillance platform suitable for educational purposes and small-scale security applications.

Table of Contents

1.0 Introduction

2.0 Materials and Equipment

2.1 Electronic Components

2.2 Equipment and Tools

3.0 Experimental Setup

4.0 Methodology

5.0 Data Collection

6.0 Data Analysis

7.0 Results

8.0 Discussion

9.0 Conclusion

10.0 Recommendations

11.0 References

1.0 Introduction

The advancement of Internet of Things (IoT) technology has transformed surveillance systems into intelligent, accessible solutions. In this lab, we will work with the ESP32-CAM module, which combines a microcontroller with an integrated camera and Wi-Fi connectivity. By integrating a servo motor with the ESP32-CAM, we can create a dynamic surveillance solution capable of autonomous or manual motion control using affordable components.

This laboratory exercise allows us to apply mechatronic system integration principles by combining multiple subsystems into a functional prototype. We will interface the ESP32-CAM with a servo motor for horizontal camera panning, establish wireless video streaming, and develop a web-based control interface. Through this project, we will gain hands-on experience with embedded systems programming, PWM-based actuator control, HTTP communication protocols, and sensor-actuator coordination.

Our implementation will progress from basic camera streaming to servo integration and manual control input. The completed system will serve as our foundation for understanding advanced topics such as motion detection and face recognition. The skills we develop are directly applicable to industries including security systems, industrial automation, and robotics, preparing us for real-world engineering challenges.

2.0 Materials and Equipment

2.1 Electronic Components

- Arduino Board – Acts as a power supply for servo motor using 5v pin.
- ESP32-cam- to read and capture video such as detecting hand gestures and light intensity
- Servo Motor - establish connections between components
- Button- Acts as a switch to turn on servo motor or turn it off.

2.2 Equipment and Tools

- Arduino IDE – To upload code for the ESP32 cam code
- USB cable – For connecting the Arduino Uno to a computer
- USB cable- For connection ESP32 cam to a computer
- Breadboard- To place button and other jumper wires.

3.0 Experimental Setup

1. Connect ESP32-CAM to the computer using USB cable.
2. Connect the servo motor to the ESP32-cam on GPIO 13 pin.
3. Place button on the breadboard and connect it to the ESP32-cam on GPIO 12 pin.
4. Power up the servo motor using external power supply like arduino uno 5v pin.
5. Place button on the breadboard.



4.0 Methodology

1. Setup the ESP32-cam
2. Setup the servo motor and button.
3. Setup the Arduino ide
4. IDE code implementation
5. Experiment the ESP32-cam with the changing in camera capture output
6. Observe the servo motor movement changes.
7. Code snippet

4.1 Detailed steps

- 1) Assemble the circuit as stated in the experimental setup.
- 2) Connect the ESP32-CAM to the computer via usb cable, and upload the code to arduino IDE.
- 3) Make sure all the Arduino Uno ground is connected together with the ground of servo motor, the ESP32-CAM and the button.
- 4) Adjust the arduino IDE code to change the intensity of the camera brightness and the movement detector value.
- 5) Observe the servo motor movement when the difference movement speed and brightness captured by the ESP32-CAM and when the button was pressed.

4.2 Programming codes

Arduino Codes

```
#include "esp_camera.h"
```

```
#include <WiFi.h>
```

```
#include "esp_http_server.h"
```

```
#include <ESP32Servo.h>
```

```
const char* ssid = "SHAHMADI_2G@unifi";
```

```
const char* password = "1234567890adi";
```

```
#define SERVO_PIN 13
```

```
#define BUTTON_PIN 12
```

```
Servo servo;
```

```
int angle = 90;
```

```
bool forward = true;
```

```
bool moveServo = false;
```

```
bool lastButtonState = HIGH;
```

```
bool buttonState = HIGH;
```

```
bool calmDetected = false;
```

```
// Camera model: AI Thinker
```

```
#define PWDN_GPIO_NUM 32
```

```
#define RESET_GPIO_NUM -1
```

```
#define XCLK_GPIO_NUM 0
```

```
#define SIOD_GPIO_NUM 26
```

```
#define SIOC_GPIO_NUM 27
```

```
#define Y9_GPIO_NUM 35
```

```
#define Y8_GPIO_NUM 34
```

```
#define Y7_GPIO_NUM 39
```

```
#define Y6_GPIO_NUM 36
```

```
#define Y5_GPIO_NUM 21
```

```

#define Y4_GPIO_NUM    19

#define Y3_GPIO_NUM    18

#define Y2_GPIO_NUM    5

#define VSYNC_GPIO_NUM 25

#define HREF_GPIO_NUM  23

#define PCLK_GPIO_NUM  22


// Variables for motion detection

uint8_t prevBrightness = 0;

int stableFrameCount = 0;

const int STABLE_THRESHOLD = 3;

const int BRIGHTNESS_CHANGE_THRESHOLD = 6;

httpd_handle_t stream_httpd = NULL;

// HTML page

static const char PROGMEM INDEX_HTML[] = R"rawliteral(

<!DOCTYPE html>

<html>

<head>

    <title>ESP32-CAM Stream</title>

    <meta name="viewport" content="width=device-width, initial-scale=1">

    <style>

        body { font-family: Arial; text-align: center; margin: 0; }

        .container { padding: 20px; }

        img { width: 100%; max-width: 640px; }

```



```
.status { margin: 20px; font-size: 18px; }

button { padding: 15px 30px; font-size: 18px; margin: 10px; cursor: pointer; }

</style>

</head>

<body>

  <div class="container">

    <h1>ESP32-CAM Control</h1>

    

    <div class="status">

      <p>Servo Status: <span id="servo">OFF</span></p>

      <p>Calm Detected: <span id="calm">NO</span></p>

    </div>

    <button onclick="toggleServo()">Toggle Servo</button>

  </div>

  <script>

    function toggleServo() {

      fetch('/toggle').then(response => response.text()).then(data => {

        document.getElementById('servo').innerText = data;

      });

    }

    setInterval(() => {

      fetch('/status').then(response => response.json()).then(data => {

        document.getElementById('servo').innerText = data.servo;

        document.getElementById('calm').innerText = data.calm;

      });

    }, 1000);

  </script>

</body>

</html>
```

```

    });

    }, 1000);

</script>

</body>

</html>

)rawliteral";

// Stream handler

static esp_err_t stream_handler(httpd_req_t *req) {

    camera_fb_t * fb = NULL;

    esp_err_t res = ESP_OK;

    size_t _jpg_buf_len = 0;

    uint8_t * _jpg_buf = NULL;

    char * part_buf[64];

    res = httpd_resp_set_type(req, "multipart/x-mixed-replace; boundary=frame");

    if(res != ESP_OK) return res;

    while(true) {

        fb = esp_camera_fb_get();

        if (!fb) {

            Serial.println("Camera capture failed");

            res = ESP_FAIL;

        } else {

```

```

if(fb->format != PIXFORMAT_JPEG) {

    bool jpeg_converted = frame2jpg(fb, 80, &_jpg_buf, &_jpg_buf_len);

    esp_camera_fb_return(fb);

    fb = NULL;

    if(!jpeg_converted) {

        Serial.println("JPEG compression failed");

        res = ESP_FAIL;

    }

} else {

    _jpg_buf_len = fb->len;

    _jpg_buf = fb->buf;

}

}

if(res == ESP_OK) {

    size_t hlen = snprintf((char *)part_buf, 64, "Content-Type: image/jpeg\r\nContent-Length:
%u\r\n\r\n", _jpg_buf_len);

    res = httpd_resp_send_chunk(req, (const char *)part_buf, hlen);

}

if(res == ESP_OK) {

    res = httpd_resp_send_chunk(req, (const char *)_jpg_buf, _jpg_buf_len);

}

if(res == ESP_OK) {

    res = httpd_resp_send_chunk(req, "\r\n--frame\r\n", 14);

}

if(fb) {

```

```

    esp_camera_fb_return(fb);

    fb = NULL;

    _jpg_buf = NULL;
} else if(_jpg_buf) {

    free(_jpg_buf);

    _jpg_buf = NULL;

}

if(res != ESP_OK) break;

}

return res;

}

// Index page handler

static esp_err_t index_handler(httpd_req_t *req) {

    httpd_resp_set_type(req, "text/html");

    return httpd_resp_send(req, (const char *)INDEX_HTML, strlen(INDEX_HTML));

}

// Toggle servo handler

static esp_err_t toggle_handler(httpd_req_t *req) {

    moveServo = !moveServo;

    Serial.print("Web toggle - Servo: ");

    Serial.println(moveServo ? "ON" : "OFF");

    httpd_resp_set_type(req, "text/plain");

```

```

    return httpd_resp_sendstr(req, moveServo ? "ON" : "OFF");
}

// Status handler

static esp_err_t status_handler(httpd_req_t *req) {
    char json[100];

    snprintf(json, sizeof(json), "{\"servo\":\"%s\",\"calm\":\"%s\"}\",
        moveServo ? "ON" : "OFF",
        calmDetected ? "YES" : "NO");

    httpd_resp_set_type(req, "application/json");
    return httpd_resp_sendstr(req, json);
}

void startCameraServer() {
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();

    config.server_port = 80;

    httpd_uri_t index_uri = {
        .uri = "/",
        .method = HTTP_GET,
        .handler = index_handler,
        .user_ctx = NULL
    };
};

```

```
httpd_uri_t stream_uri = {  
  
    .uri = "/stream",  
  
    .method = HTTP_GET,  
  
    .handler = stream_handler,  
  
    .user_ctx = NULL  
  
};
```

```
httpd_uri_t toggle_uri = {  
  
    .uri = "/toggle",  
  
    .method = HTTP_GET,  
  
    .handler = toggle_handler,  
  
    .user_ctx = NULL  
  
};
```

```
httpd_uri_t status_uri = {  
  
    .uri = "/status",  
  
    .method = HTTP_GET,  
  
    .handler = status_handler,  
  
    .user_ctx = NULL  
  
};
```

```
if (httpd_start(&stream_httpd, &config) == ESP_OK) {  
  
    httpd_register_uri_handler(stream_httpd, &index_uri);  
  
    httpd_register_uri_handler(stream_httpd, &stream_uri);  
  
}
```

```

    httpd_register_uri_handler(stream_httpd, &toggle_uri);

    httpd_register_uri_handler(stream_httpd, &status_uri);

    Serial.println("Camera server started");
}

}

void setup() {

    Serial.begin(115200);

    Serial.println("ESP32-CAM Calm Detection + Button Control");

    pinMode(BUTTON_PIN, INPUT_PULLUP);

    servo.attach(SERVO_PIN);

    servo.write(angle);

    camera_config_t config;

    config.ledc_channel = LEDC_CHANNEL_0;

    config.ledc_timer = LEDC_TIMER_0;

    config.pin_d0 = Y2_GPIO_NUM;

    config.pin_d1 = Y3_GPIO_NUM;

    config.pin_d2 = Y4_GPIO_NUM;

    config.pin_d3 = Y5_GPIO_NUM;

    config.pin_d4 = Y6_GPIO_NUM;

    config.pin_d5 = Y7_GPIO_NUM;

    config.pin_d6 = Y8_GPIO_NUM;

```

```
config.pin_d7 = Y9_GPIO_NUM;

config.pin_xclk = XCLK_GPIO_NUM;

config.pin_pclk = PCLK_GPIO_NUM;

config.pin_vsync = VSYNC_GPIO_NUM;

config.pin_href = HREF_GPIO_NUM;

config.pin_sscb_sda = SIOD_GPIO_NUM;

config.pin_sscb_scl = SIOC_GPIO_NUM;

config.pin_pwdn = PWDN_GPIO_NUM;

config.pin_reset = RESET_GPIO_NUM;

config.xclk_freq_hz = 20000000;

config.pixel_format = PIXFORMAT_JPEG; // Changed to JPEG for streaming

config.frame_size = FRAMESIZE_VGA; // Better quality

config.jpeg_quality = 10;

config.fb_count = 2;


if (esp_camera_init(&config) != ESP_OK) {

    Serial.println("Camera init failed!");

    while (true);

}


WiFi.begin(ssid, password);

Serial.print("Connecting to WiFi");

while (WiFi.status() != WL_CONNECTED) {

    delay(300);
```



```

    Serial.print(".");

}

Serial.println("\nWiFi connected!");

Serial.print("Camera Stream URL: http://");

Serial.println(WiFi.localIP());


startCameraServer();

}


void loop() {

    // Button control

    buttonState = digitalRead(BUTTON_PIN);

    if (buttonState == LOW && lastButtonState == HIGH) {

        delay(50);

        buttonState = digitalRead(BUTTON_PIN);

        if (buttonState == LOW) {

            moveServo = !moveServo;

            Serial.print("Button pressed - Servo: ");

            Serial.println(moveServo ? "ON" : "OFF");

        }

    }

    lastButtonState = buttonState;

    // Camera calm detection (periodic check, not every loop)

    static unsigned long lastCheck = 0;

```

```

if (millis() - lastCheck > 200) { // Check every 200ms

    lastCheck = millis();

    camera_fb_t* fb = esp_camera_fb_get();

    if (fb) {

        uint32_t total = 0;

        for (int i = 0; i < fb->len; i += 10) { // Sample every 10th pixel

            total += fb->buf[i];

        }

        uint8_t avgBrightness = total / (fb->len / 10);

        esp_camera_fb_return(fb);

        int brightnessChange = abs(avgBrightness - prevBrightness);

        if (brightnessChange < BRIGHTNESS_CHANGE_THRESHOLD) {

            stableFrameCount++;

            if (stableFrameCount >= STABLE_THRESHOLD && !calmDetected) {

                calmDetected = true;

                Serial.println("CALM detected - Servo STOPPED");

            }

        } else {

            stableFrameCount = 0;

            if (calmDetected) {

                calmDetected = false;

```

```

        Serial.println("Movement detected");
    }
}

prevBrightness = avgBrightness;
}
}

// Servo control

if (moveServo && !calmDetected) {
    if (forward) angle++;
    else angle--;
    if (angle >= 180) forward = false;
    if (angle <= 0) forward = true;
    servo.write(angle);
    delay(20);
} else {
    delay(20);
}
}

```

5.0 Data Collection

In order to assess the effectiveness of the Smart Surveillance System constructed using the ESP32-CAM module, data was gathered throughout this experiment. Four key components of the system were the focus of the data gathering process: pushbutton control behaviour, servo motor functioning, video streaming performance, and Wi-Fi connection. Initially, the time it took for the ESP32-CAM to connect to the Wi-Fi network was noted, together with the IP address shown in the Serial Monitor and the stability of the connection throughout continuous operation. The average frame rate, clarity, and latency of the live video feed were then noted while watching the video streaming output through the web browser. Additionally, information on the

servo motor's horizontal panning was gathered by recording its minimum and maximum PWM values, the approximate range of rotation attained, and the amount of time it took for the servo to do a full sweep. Lastly, the responsiveness and stability of the digital input with the internal pull-up setup were noted, and the pushbutton input was tested to see if the servo moved only when the button was pushed. Real-time observations and system output measurements served as the foundation for all of the data that was gathered.

6.0 Data Analysis

The ESP32-CAM system was able to carry out all necessary tasks with generally steady behaviour, according to an analysis of the data gathered. The module needed a brief but acceptable amount of time to establish its network connection, as seen by the Wi-Fi connection time, which averaged between three and six seconds. Although there were a few slight delays when the signal intensity changed, the Wi-Fi connection was largely constant. The camera provided a live feed for video streaming at a rate of about 12–18 frames per second, which is suitable for simple surveillance. Due to picture compression and network transmission, there was a slight lag between the actual scene and the streaming video, typically 200–400 ms, but the stream remained uninterrupted and clear. With PWM values between 3700 and 7000, the servo motor demonstrated smooth and steady panning over an estimated range of 0° to 180°. When driven by a steady 5V 2A source, its performance significantly increased, guaranteeing steady movement and minimising jitter. Because the servo only moved when the button was pressed and the reaction time was almost quick, the pushbutton control worked well. This attests to the system's successful integration of manual input. Overall, the research shows that the pushbutton, servo motor, and ESP32-CAM were correctly synchronised to create a working IoT-based surveillance system.

7.0 Results

The experiment's findings demonstrate that the Smart Surveillance System prototype functioned effectively and met all of its goals. The ESP32-CAM produced a steady VGA-quality feed with a respectable frame rate by successfully connecting to Wi-Fi and streaming live video to a web interface. The servo motor demonstrated efficient actuator control by smoothly executing horizontal panning in accordance with the specified PWM values. Once the pushbutton was added, the servo only reacted when the button was pressed, indicating that the input–actuator integration was done correctly. Throughout the testing phase, the system demonstrated quick responsiveness, steady performance, and consistent behaviour. Overall, the experiment confirmed that the ESP32-CAM can operate as a straightforward yet efficient smart surveillance system appropriate for simple monitoring applications when combined with servo control and human triggering.

8.0 Discussion

This experiment integrates the ESP32-CAM module, which allows users to stream online video while connected to the servo motor. It needs to use the same Wi-Fi network to view the video. The included HTML interface allows for easy control of the servo motor, giving users the ability to change its movement remotely. This demonstrates how an embedded system can combine networking, video streaming, and mechanical actuation in a single compact device.

In addition to user-controlled movement, the system also includes basic still motion detection. The ESP32-CAM can be used to check for changes in image brightness to determine whether the scene is still or moving. When the environment remains still for a few frames, the code automatically stops the servo and prints a message indicating still detection. It demonstrates how simple algorithms can be used to interpret camera data and trigger mechanical responses.

Finally, physical buttons offer an additional method of controlling the servo motor. Pressing the button will turn the servo on or off and no movement will be sent to the servo motor. Even though there is still movement on the ESP32-CAM, the servo motor is set to not make any movement at all. This experiment highlights how the ESP32-CAM can be used to combine live video streaming, servo motor control, and basic motion detection in one integrated system.

9.0 Conclusion

This experiment successfully demonstrated the ESP32-CAM's ability to integrate video streaming, servo motor control, and basic motion detection into a working IoT system. The results show that the device can not only provide real-time camera feedback via a web interface but also intelligently respond to environmental changes by stopping the servo motor even when there is movement. The addition of a physical button further strengthens the usability of the system and offers an alternative way to control the servo. Overall, this experiment proves that the ESP32-CAM is a powerful and versatile platform for developing smart monitoring and automation applications.

10.0 Recommendation

It is advisable for users to be reminded that after uploading or displaying the code, the IO0 pin must be disconnected from GND immediately. If it is not removed quickly, the ESP32-CAM will fail to boot normally.

For future improvements, it is recommended to improve the motion detection algorithm by using pixel-based comparison or AI-assisted detection for higher accuracy. In addition, the web interface should also include password protection to increase security, especially when connected to shared or public Wi-Fi. Finally, the integration of additional sensors such as PIR motion sensors or sound sensors can improve reliability and reduce false detections. This will make the system more advanced and convincing.

12.0 Acknowledgement

Special thanks to ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO for their guidance and support during this experiment.

Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Name: ADAM NURUDDIN BIN HELMI

Read []

Matric Number: 2215783

Understand []

Signature: *ADAM*

Agree []

Name: NUR FATIHAH HANNANI BINTI HASMAYADI

Read []

Matric Number: 2227062

Understand []

Signature: *FAT*

Agree []

Name: NUR SYAHZMAN AZIQ BIN MOHD SHAHMADI

Read []

Matric Number: 2310037

Understand []

Signature: *AZIQ*

Agree []